

Introduction of our DSA Visualization

1. How to Use the Program

The String Matching Visualizer (KMP) is an interactive tool designed to demonstrate how the Knuth-Morris-Pratt algorithm efficiently searches for a pattern within a text string.

Step-by-Step Instructions:

1. **Input the Text String:** (length less than 14)
 2. **Input the Pattern String:** (text length less than 14)
 3. **Preprocessing Phase:** The program will automatically calculate and display the prefix function step-by-step using a rabbit mascot to track the matching prefixes and suffixes (representing k).
 4. **Main Process Phase:** The visualizer will simulate the string matching process. Green highlighted boxes indicate matching characters, while the program demonstrates how the KMP algorithm skips redundant comparisons based on the pi function table.
 5. **Results Panel:** Once a match is found, the program highlights the matching segment in the text. When all the process is over, the matched shifts will be shown in the side panel (Match-Shift list).
-

2. Data Structure and Algorithm Explanation

The program visualizes the Knuth-Morris-Pratt (KMP) Algorithm, a highly efficient linear-time string-matching algorithm with a time complexity of $O(m+n)$, where “ n ” is the length of the text and “ m ” is the length of the pattern. Unlike the naive approach, KMP avoids rolling back the text pointer when a mismatch occurs by utilizing precomputed shift information.

Core Logic: The Prefix Function

The core data structure used is a 1D array called the pi table (or failure function). For each position “ i ” in the pattern, “ $pi[i]$ ” stores the length of the longest proper prefix of

the pattern block that is also a suffix of that block (of course its length should be shorter than i).

Step-by-Step Example

Phase 1: Preprocessing the Pattern

- **Pattern:** a b a
- **pi Table Derivation:**
 - For a (index 0): No proper prefix/suffix. $\text{pi}[0] = 0$.
 - For a b (index 1): No matching prefix and suffix. $\text{pi}[1] = 0$.
 - For a b a (index 2): The proper prefix a matches the suffix a. Length is 1. $\text{pi}[2] = 1$.

The resulting pi table is:

Index	0	1	2
Pattern	a	b	a
Pi.value	0	0	1

Phase 2: Main Matching Process

- **Text:** a b a a b c a a
 - **Pattern:** a b a
1. **First Match Attempt:** The algorithm compares the text and pattern from left to right.
 - It successfully matches the first three characters: a, b, and a.
 - **Result:** A full match is found at shift 0!!!!
 2. **Shifting by the Pi Table:**
 - Instead of shifting by just 1 character and re-checking everything from scratch, KMP looks at the last successfully matched character's pi value ($\text{pi}[2] = 1$).
 - This tells the algorithm that the last character a can also serve as the beginning prefix of the next potential match.
 - The pattern safely shifts forward, skipping unnecessary comparisons and keeping the text scanning pointer moving strictly forward.